Secretos en RAM: memoria volátil y no volátil

Proyecto 3 – Grupo 3

Raúl Quesada Morera, Justin Rodríguez González, Cristian Rojas Morales y Keilor Madrigal Ramírez

Universidad Técnica Nacional

ITI922 – Seguridad de TI II

Investigación de la Unidad III

Marcos Andrés Castro Calvo

28 de octubre de 2026

Tabla de contenidos

Introducción 3
Marco teórico 3
Memoria volátil y no volátil 3
Ataque de arranque en frío (*cold boot*) 4
Llaves de cifrado de disco en la RAM 5
Riesgos de la *swap*, la hibernación y los volcados de memoria 6
Memoria volátil y no volátil dentro del TPM 6
Buenas prácticas de programación 7
Caso real 7
Descripción de la práctica 8
Arquitectura y herramientas 8
Programas en C y ataque 9
Experimento con Python 10
Volcado por falla del programa 10
Experimento volátil y no volátil con el TPM 11
Análisis 11
Conclusiones y recomendaciones 12
Referencias 12
Anexo A. Bitácora de uso de inteligencia artificial 15

Secretos en RAM: memoria volátil y no volátil

Introducción

Un secreto protegido en disco deja de estar protegido cuando se usa: para que una contraseña o una llave cumplan su función, el programa debe descifrarla y copiarla en la memoria principal (RAM), donde queda en texto claro hasta que alguien la borre. Esos datos pueden recuperarse con un volcado de memoria, un archivo de intercambio (*swap*), una imagen de hibernación o un ataque de arranque en frío. Además, no toda la memoria se comporta igual: la RAM pierde su contenido al apagarse, aunque más despacio de lo que se cree, mientras que el disco, la memoria flash y la memoria no volátil de un TPM lo conservan.

El grupo respondió las siete preguntas del proyecto, analizó el caso Heartbleed y el análisis forense de memoria, y construyó una práctica en Ubuntu 24.04 con programas en C, un script en Python, un volcado por falla y un TPM emulado. Los ataques se hicieron solo sobre datos creados por el grupo.

Marco teórico

Memoria volátil y no volátil

La memoria se organiza en una jerarquía que equilibra velocidad, capacidad y persistencia: los registros y la caché responden en nanosegundos, la memoria principal (DRAM) en decenas de nanosegundos y el disco en milisegundos (García Carballeira y Calderón Mateos, s. f.). Un medio es volátil si necesita energía para conservar los datos, como la RAM y la caché; la ROM y la memoria flash no la necesitan. Pero el programador no decide dónde acaba un dato: cuando falta RAM, el sistema operativo expulsa páginas al área de intercambio (*swap*) del disco (Carretero Pérez et al., s. f.). La Tabla 1 detalla cuánto dura el dato en cada nivel, qué riesgo corre y cómo se protege.

Tabla 1
Qué sobrevive en cada tipo de memoria, su riesgo y su protección

Memoria y duración del dato	¿Sobrevive al reinicio?	Riesgo principal	Protección habitual
Caché de CPU: nanosegundos, hasta que se reemplaza la línea	No	Bajo: el dato se reemplaza enseguida y desaparece al apagar	Se protege la RAM de la que proviene
RAM (DRAM): mientras hay energía; segundos a minutos tras cortarla	No, pero con remanencia	Arranque en frío y volcados de memoria	Borrado explícito, apagado completo, cifrado de memoria
Swap: hasta que otra página la sobrescribe	Sí	Páginas con secretos escritas en el disco	`mlock` y *swap* cifrada
Hibernación: mientras exista la imagen	Sí	Copia completa de la RAM en el disco	Volumen cifrado; apagar en lugar de hibernar
Volcado (*core dump*): mientras exista el archivo	Sí	Memoria del proceso en un archivo legible	Desactivar volcados; permisos estrictos
Disco, SSD y memoria flash: mientras no se sobrescriban	Sí	Robo o copia del medio	Cifrado de disco (LUKS, BitLocker)
NVRAM del TPM: hasta que se elimine el objeto	Sí	Lectura con autorización indebida	Políticas de PCR y contraseña

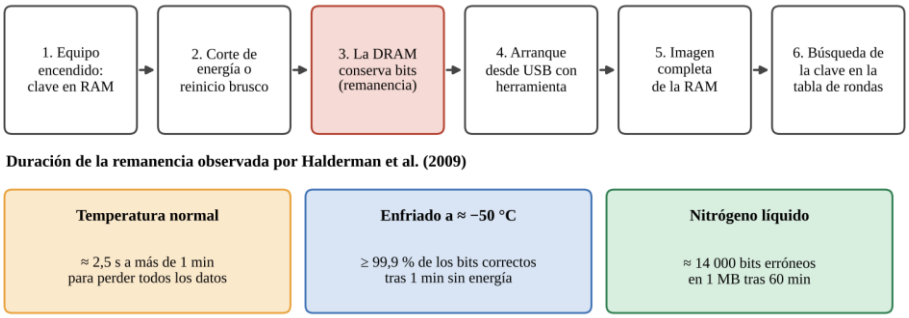
Ataque de arranque en frío (*cold boot*)

El ataque de arranque en frío (*cold boot*) aprovecha la remanencia de la DRAM: sus celdas pierden la carga poco a poco al cortar la energía. Quirantes Sierra (2008) recuerda que se creía que la RAM se borraba casi enseguida, pero que módulos de varios fabricantes conservaron los datos entre 2 y 50 segundos. Halderman et al. (2009) midieron que a temperatura normal la pérdida total tardó entre unos 2,5 segundos y más de un minuto; con los chips enfriados a cerca de −50 °C se conservó al menos 99,9 % de los bits tras un minuto sin energía y, con nitrógeno líquido, solo hubo unos 14 000 bits erróneos en 1 MB tras 60 minutos.

Como muestra la Figura 1, el atacante reinicia el equipo bruscamente, arranca un sistema mínimo desde un USB, copia la RAM y busca llaves. Así se recuperaron llaves de BitLocker, FileVault, TrueCrypt y dm-crypt, incluso con bits degradados (Halderman et al., 2009). Las medidas conocidas no eliminan el riesgo; la más sencilla es apagar del todo los equipos sensibles.

Figura 1

Etapas del ataque de arranque en frío y duración de la remanencia

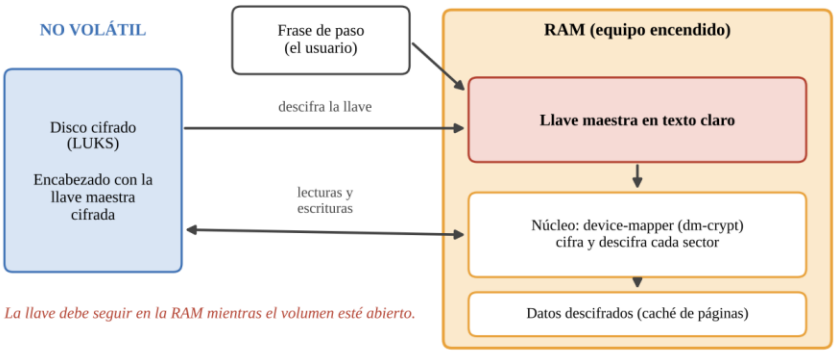


Llaves de cifrado de disco en la RAM

Cifrar el disco con LUKS o BitLocker protege los datos en reposo, pero el sistema debe descifrarlos al usarlos. En LUKS, la frase de paso desbloquea una llave maestra almacenada cifrada en el encabezado; una vez descifrado el dispositivo, los archivos quedan disponibles para quien normalmente tendría acceso (Red Hat, s. f.). Como LUKS usa el subsistema *device-mapper* del núcleo, cada lectura y escritura pasa por el núcleo, que cifra o descifra cada sector con esa llave; por eso la llave debe permanecer en la RAM mientras el volumen esté abierto (Figura 2). Halderman et al. (2009) recuperaron llaves de equipos encendidos o recién apagados y advierten que el TPM no protege una llave ya cargada en la RAM.

Figura 2

Camino de la llave maestra de un volumen LUKS



Riesgos de la *swap*, la hibernación y los volcados de memoria

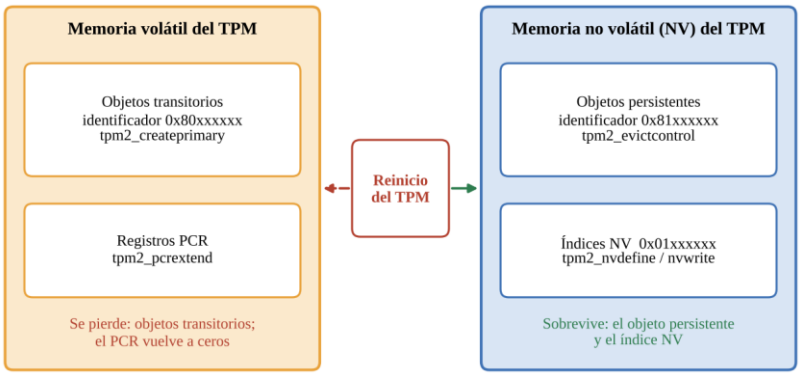
La *swap* guarda en disco las páginas expulsadas de la RAM, por lo que una contraseña puede sobrevivir al apagado. La hibernación guarda una imagen completa de la RAM, con las llaves en uso; por eso Halderman et al. (2009) aconsejan apagar por completo en lugar de suspender o hibernar. Un volcado de memoria (*core dump*) es un archivo con la memoria de un proceso, que se genera cuando el programa falla o a propósito con herramientas como `gcore`; la entrada CWE-14 advierte que así puede recuperarse una contraseña que quedó en memoria (MITRE, s. f.).

Memoria volátil y no volátil dentro del TPM

Un TPM 2.0 maneja datos con distintos ciclos de vida (Trusted Computing Group, 2024). Los *objetos transitorios* (identificadores 0x80xxxxxx) viven en la memoria volátil del TPM y se pierden al reiniciar. Los *objetos persistentes* (0x81xxxxxx) se copian a la memoria no volátil con `tpm2_evictcontrol` y sobreviven. Los *índices NV* (0x01xxxxxx) son áreas de datos no volátiles, con atributos y políticas de autorización propios. Los registros PCR son volátiles: tras un arranque limpio vuelven a su valor inicial (Figura 3).

Figura 3

Objetos del TPM 2.0 según su memoria y su comportamiento al reiniciar



Buenas prácticas de programación

Un programa debe reducir el tiempo que un secreto vive en memoria y evitar copiarlo. Sobrescribirlo con `memset` puede ser inútil: si el compilador detecta que el búfer no se vuelve a leer, puede eliminar la llamada como escritura muerta (*dead store*) (MITRE, s. f.); por eso C ofrece `explicit_bzero`. `mlock` bloquea páginas en la RAM e impide que vayan a la *swap*, pero no impide leer el secreto mientras se usa. En Python las cadenas (`str`) son inmutables: no se pueden sobrescribir y `del` solo suprime la referencia. Un `bytearray` sí es mutable, pero ayuda solo si el secreto nunca pasó por un objeto inmutable como `bytes` o `str`.

Caso real

Heartbleed (CVE-2014-0160) es una vulnerabilidad de OpenSSL en la extensión *heartbeat* de TLS. Según Knudsen (2021), surgió al actualizar OpenSSL para TLS 1.2 en 2012: el mensaje de latido incluye un campo de longitud y una carga que el servidor debe repetir, pero OpenSSL confiaba en la longitud declarada por el cliente. Si la solicitud mentía, el servidor devolvía memoria contigua. Cada solicitud leía hasta 64 kB sin dejar rastro y expuso llaves privadas, credenciales y otros datos; las medidas fueron actualizar OpenSSL, reemitir certificados y cambiar contraseñas. Fue una fuga de información, no un descifrado del tráfico, y se descubrió mediante *fuzzing*.

La lección es doble. Un secreto en uso tiene que estar en la RAM, y un defecto que permita leerla lo entrega sin romper el cifrado: borrar la variable no habría protegido las llaves de un servidor que las necesita en cada conexión, y por eso hay que rotar llaves, aislarlas en otro proceso o módulo de hardware y parchear rápido. Además, hay que conocer y probar el código de terceros, no confiar en que el proveedor lo hizo bien.

El uso defensivo de la misma idea es el análisis forense. Mendillo (s. f.) explica que algunas intrusiones no dejan rastros en el disco y que un volcado permite un análisis repetible y

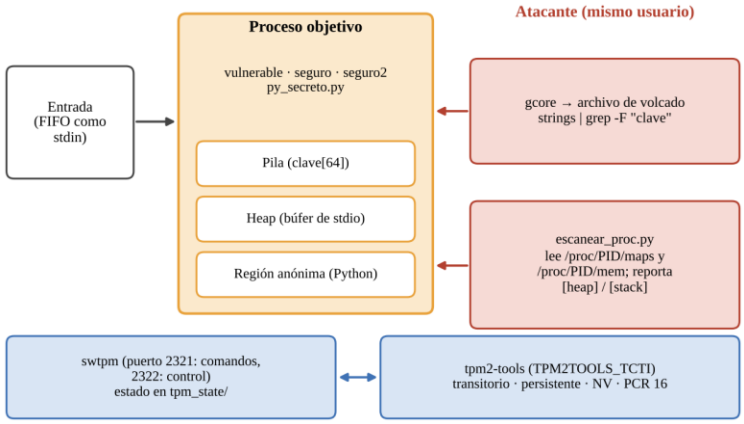
menos intrusivo que el análisis en vivo, ya sea básico, con *strings*, o avanzado, con Volatility. Nuestro ataque con `strings` y `grep` equivale al nivel básico.

Descripción de la práctica

Arquitectura y herramientas

La práctica se ejecutó en Ubuntu 24.04 LTS con `gcc`, `gdb` (incluye `gcore`), `strings`, `grep`, Python 3, `swtpm` y `tpm2-tools` 5.6. El dato atacado es una contraseña de prueba creada por el grupo (`ClaveSecreta2026!`). Cada programa recibe la contraseña por una FIFO y queda en pausa; `gcore` vuelca su memoria, `strings` y `grep` la examinan, y un guion lee `/proc/PID/mem` para saber en qué región quedó cada copia (Figura 4). La lista de programas, el código y los pasos para reproducirlos están en el Anexo B. Para que `gcore` y la lectura de `/proc/PID/mem` funcionaran sin privilegios de administrador se fijó `kernel.yama.ptrace_scope=0` (Ubuntu trae el valor 1, que solo permite depurar procesos hijos), y para que los volcados por falla se escribieran en la carpeta de trabajo, en lugar de enviarse a apport, se fijó `kernel.core_pattern=core.%p`. Ambos cambios se aplican con `preparar.sh` y se revierten con `restaurar.sh`. La práctica se volvió a ejecutar completa el 27 de octubre de 2026 y todos los resultados coincidieron con los de las Tablas 2 y 3.

Figura 4
Arquitectura de la práctica



.

Programas en C y ataque

El programa vulnerable lee la contraseña con `fgets` y no la borra. Al volcar su memoria, `strings` y `grep` la encontraron en dos lugares: la variable `clave[64]` en la pila y una copia en el búfer interno de `stdio`, en el *heap* (Figura 5 y Tabla 2). Esa copia nace porque `fgets` lee a través de un búfer de la biblioteca (Figura 6). La posición de la copia en la pila cambió en cada ejecución por la aleatorización del espacio de direcciones (ASLR); la del *heap* se mantuvo cerca del byte 18 000 del volcado.

Figura 5

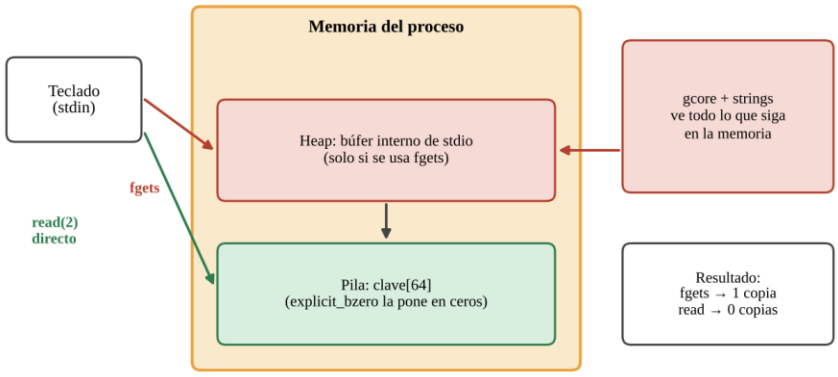
Salida del ataque contra el programa vulnerable y contra la versión corregida



Reproducción de la salida de la terminal en Ubuntu 24.04. En rojo: la contraseña fue encontrada.

El resultado más instructivo fue el de `seguro.c`: `explicit_bzero` puso `clave[64]` en ceros y `mlock` bloqueó 4 kB, pero el volcado aún contenía la copia de `stdio`. Solo `seguro2.c`, que lee con `read(2)`, no dejó rastro. No basta borrar la variable visible: hay que controlar todas las copias intermedias.

Figura 6
Copias de la contraseña en la memoria de un programa en C



.

Experimento con Python

El script `py_secreto.py` lee la contraseña, espera, intenta borrarla y cuenta sus apariciones en la memoria viva del proceso antes y después. Con `str` hubo tres copias antes y después de `del` y `gc.collect()`. Con `bytes_a_bytearray` la sobrescritura bajó las copias de tres a dos: cayó la mutable y quedaron las inmutables. Con `bytearray_directo` (`os.readv` escribe en el `bytearray`) la única copia desapareció al ponerla en ceros.

Volcado por falla del programa

Para observar el riesgo de los volcados por falla se ejecutaron `vulnerable` y `seguro2` con `ulimit -c unlimited` y se les envió `SIGSEGV`. Cada caso generó un archivo `core` de 450 560 bytes con permisos `-rw-------`: el de `vulnerable` contenía la contraseña dos veces y el de `seguro2` ninguna; con `ulimit -c 0` no se generó archivo (Tabla 2).

Tabla 2
Resultados de los ataques contra los programas en C, en Python y contra volcados por falla

Caso	Copias encontradas	Región de memoria	Páginas bloqueadas
C vulnerable (`fgets`, sin borrar)	2	`[heap]` y `[stack]`	0 kB
C seguro (`fgets` + `mlock` + `explicit_bzero`)	1	`[heap]` (búfer de `stdio`)	4 kB
C seguro2 (`read` + `mlock` + `explicit_bzero`)	0	Ninguna	4 kB
Python `str`	3 → 3	`[heap]` (1) y anónimas (2)	No aplica
Python `bytes_a_bytearray`	3 → 2	`[heap]` (1) y anónima (1)	No aplica

Caso	Copias encontradas	Región de memoria	Páginas bloqueadas
Python `bytearray_directo`	1 → 0	Anónima, luego ninguna	No aplica
Volcado por falla de `vulnerable`	2	Archivo `core`	No aplica
Volcado por falla de `seguro2`	0	Archivo `core`	No aplica

Experimento volátil y no volátil con el TPM

Con `swtpm` se creó una clave primaria transitoria, otra copiada a la memoria persistente (0x81010001), un índice NV (0x01500016) con el texto «secreto-en-NV-2026» y se extendió el PCR 16 con el hash de «malware». Luego se apagó el TPM (`tpm2_shutdown -c`) y se reinició (`tpm2_startup -c`). Sobrevivieron el objeto persistente y el índice NV; se perdieron el transitorio y el PCR (Tabla 3).

Tabla 3
Estado de los objetos del TPM antes y después de reiniciarlo

Elemento	Tipo	Antes del reinicio	Después del reinicio
Clave primaria (`tpm2_createprimary`)	Transitorio	Tres identificadores, 0x80000000 a 0x80000002	Lista vacía; cargar el contexto guardado falla con un error del TPM
Clave copiada con `tpm2_evictcontrol`	Persistente (0x81010001)	Presente	Presente; `tpm2_readpublic` devuelve su nombre
Índice NV 0x01500016	NV	Lectura: «secreto-en-NV-2026»	Lectura idéntica
PCR 16 extendido con «malware»	Volátil	F54326D9…5221266F	Todo en ceros

Análisis

La práctica demostró cuatro ideas que confirman la teoría. Un secreto sin borrar se recupera con herramientas básicas, como Halderman et al. (2009) hicieron con llaves de disco y Heartbleed permitió a distancia (Knudsen, 2021). Corregir exige más que una llamada: `explicit_bzero` y `mlock` no bastaron por la copia de `stdio`. En Python, la inmutabilidad de `str` impide el borrado. Y la diferencia entre memoria volátil y no volátil es observable: el TPM conservó el índice NV y el objeto persistente y perdió el transitorio y el PCR.

Los límites son varios. El ataque fue local, con el mismo usuario y con `ptrace_scope=0`; se probó una sola contraseña; el TPM es un emulador sin resistencia física; y no se midieron la *swap*, la hibernación ni el arranque en frío, que se sustentan solo en las fuentes. En un entorno real se delegarían las llaves a un TPM o módulo de hardware, se desactivarían los volcados (`RLIMIT_CORE=0`), se protegería la *swap*, se apagarían los equipos sensibles (Halderman et al., 2009) y este ataque se usaría como prueba automática.

Conclusiones y recomendaciones

Raúl Quesada Morera.

La investigación mostró que un dato protegido en disco deja de estarlo cuando se usa, porque debe copiarse en la RAM, y que esa memoria no se borra de inmediato al perder la energía (Halderman et al., 2009). Por eso conviene apagar por completo los equipos sensibles y no solo suspenderlos o hibernarlos.

Justin Rodríguez González.

La práctica confirmó que proteger un secreto no es solo borrar una variable: el programa puede dejar copias intermedias en otras zonas de memoria y un volcado las revela. Se recomienda reducir el tiempo de vida de los secretos, evitar copiarlos y comprobar con un volcado que no quedan rastros.

Cristian Rojas Morales.

El lenguaje de programación influye en la seguridad: con datos inmutables no se puede sobrescribir el secreto, y un programa que falla puede entregar toda su memoria en un volcado. Se recomienda usar estructuras que permitan borrar el dato, desactivar los volcados innecesarios y delegar lo más crítico a componentes especializados.

Keilor Madrigal Ramírez.

Distinguir entre memoria volátil y no volátil permite decidir qué se guarda y dónde: en un TPM lo transitorio se pierde al reiniciar y lo persistente sobrevive. Casos como Heartbleed muestran que leer la memoria basta para exponer llaves (Knudsen, 2021). Se recomienda guardar en

memoria no volátil solo lo necesario, con controles de acceso, y rotar las llaves tras un incidente.

Referencias

Carretero Pérez, J., Calderón Mateos, A., García Sánchez, J. D., García Blas, F. J., Pérez Lobato, J. M., y Casares Andrés, M. G. (s. f.). *Sistemas operativos: Lección 10. Gestión de memoria virtual* [Material de clase]. OpenCourseWare, Universidad Carlos III de Madrid. https://ocw.uc3m.es/pluginfile.php/3335/mod_page/content/19/memoria_virtual.pdf

García Carballeira, F., y Calderón Mateos, A. (s. f.). *Tema 5. Jerarquía de memoria* [Material de la asignatura Estructura de Computadores]. OpenCourseWare, Universidad Carlos III de Madrid. https://ocw.uc3m.es/pluginfile.php/1917/mod_page/content/19/intro_jerarquia_memoria.pdf

Halderman, J. A., Schoen, S. D., Heninger, N., Clarkson, W., Paul, W., Calandrino, J. A., Feldman, A. J., Appelbaum, J., y Felten, E. W. (2009). Lest we remember: Cold-boot attacks on encryption keys. *Communications of the ACM, 52*(5), 91–98. https://doi.org/10.1145/1506409.1506429

Knudsen, J. (2021, 7 de diciembre). *Ciberseguridad: un análisis profundo de Heartbleed*. DatacenterDynamics. https://www.datacenterdynamics.com/es/opinion/ciberseguridad-un-an%C3%A1lisis-profundo-de-heartbleed/

Mendillo, V. (s. f.). *Análisis forense de la memoria RAM* [Ponencia]. Facultad de Ingeniería, Universidad Central de Venezuela. https://www.ing.ucv.ve/jifi2018/documentos/electronica/conferencias/CM-ERT-10.pdf

MITRE. (s. f.). *CWE-14: Compiler removal of code to clear buffers* (versión 4.20) [Entrada de base de datos]. Common Weakness Enumeration. Recuperado el 14 de octubre de 2026, de https://cwe.mitre.org/data/definitions/14.html

Quirantes Sierra, A. (2008, 2 de marzo). Ataques fríos contra ordenadores calientes. *Boletín ENIGMA, 58*. Taller de Criptografía. https://www.ugr.es/~aquiran/cripto/enigma/boletin_enigma_58.htm

Red Hat. (s. f.). Capítulo 8. Cifrado de dispositivos de bloque mediante LUKS. En *Red Hat Enterprise Linux 8: Endurecimiento de la seguridad*. Red Hat Documentation. https://docs.redhat.com/es/documentation/red_hat_enterprise_linux/8/html/security_hardening/encrypting-block-devices-using-luks_security-hardening

Trusted Computing Group. (2024). *Trusted Platform Module library specification, family "2.0", level 00, revision 01.83*. https://trustedcomputinggroup.org/?p=12014

Anexo A
Bitácora de uso de inteligencia artificial

Tabla 1

Consultas hechas a la IA durante la elaboración del informe y demo

Fecha	Herramienta	Qué le pedimos	Qué respondió (resumen)	Qué verificamos o corregimos
14/10/2026	Claude (Anthropic)	Ayuda para estructurar el informe del Proyecto 3 en formato APA 7 a partir de la guía de la investigación, con todos los apartados pedidos, y una versión de avance.	Leyó la guía, identificó las siete preguntas de «Qué deben investigar» y propuso la estructura del informe completo y del avance, con un borrador del texto.	El grupo comparó la estructura con la guía.
14/10/2026	Claude (Anthropic)	Ayuda para buscar referencias APA (mínimo 6, al menos 3 oficiales o académicas) y, después, que fueran en español.	Buscó y propuso fuentes candidatas.	Se abrió cada fuente para confirmar autores, año e idioma. Se descartaron el blog de Microsoft, páginas man traducidas (mezcladas con inglés) y fuentes sin autor o fecha. Se aceptaron 3 fuentes en inglés (Halderman et al., TCG y MITRE) por no hallar equivalente en español.
18/10/2026	Claude (agente en la VM)	Ayuda para armar la demo en una VM Ubuntu 24.04 a partir del Anexo B.	Propuso los scripts por paso y los dos escáneres que faltaban, y ajustó `preparar.sh` para no sobrescribir los valores originales.	Se ejecutó la práctica tres veces; los resultados coincidieron con las Tablas 2 y 3 y se actualizaron los tamaños de los volcados.
27/10/2026	Claude (Anthropic)	Ayuda para revisar si el informe cumple la guía del proyecto.	Señaló que faltaban la caché en la Tabla 1, la configuración del sistema sin documentar y scripts ausentes en el Anexo B.	Se comprobó cada punto contra la guía y se corrigió en esta versión.